Format-Aware Checkpoint Loading: An Adaptive Backend Policy for LLMs

Botir Khaltaev
Independent, United Kingdom
btrghstk@gmail.com

April 2026

Abstract

Inference on large language models requires loading weights from storage into GPU memory. In serverless settings where idle models are evicted, cold-start latency hinges on how checkpoints are read. Peak NVMe bandwidth is only an upper bound: checkpoint layout, request structure, and integration overhead often dominate realised throughput.

This paper treats checkpoint loading as a measurement-driven systems problem. Using Tensora, we compare synchronous POSIX I/O, Tokio async, io_uring, and an adaptive `default` policy across contiguous SafeTensors and partitioned ServerlessLLM-style layouts, then trace those choices through Python bindings and vLLM integration.

For contiguous SafeTensors, explicit sync wins at smaller scales while explicit io_uring wins at the largest multi-shard scales where queue depth matters. For partitioned ServerlessLLM, sync never leads; the relevant trade-off is among async, io_uring, and `default`. These layout- and scale-dependent crossovers show that no single fixed backend is optimal in all cases.

Deploy a format-aware adaptive `default` with explicit overrides, not a single universal loader. Python and vLLM results confirm that integration overhead can reorder backend rankings; loader choice matters most for initialisation and time-to-first-token.

1 Introduction

Serving an LLM begins before the first token: weights must reach GPU memory. A 7B model in float16 needs roughly 14GB, and serverless environments evict idle models to free memory for other tenants. When a request arrives, the inference pipeline cannot start until the checkpoint is fully loaded—this cold-start latency is directly visible to users.

Peak storage bandwidth is only an upper bound. Checkpoint layout, request structure, and integration overhead often dominate realised throughput. The gap between peak hardware and effective loading performance suggests bottlenecks lie in how reads are submitted, batched, and integrated.

Our thesis is that backend choice for LLM checkpoint loading depends jointly on layout and scale, so the right systems contribution is a measured, format-aware adaptive policy with explicit overrides, not a single universal loader. We use Tensora as the experimental vehicle, comparing synchronous POSIX I/O, Tokio async, io_uring, and an adaptive `default` under cold-cache evaluation.[1]

For SafeTensors (contiguous), explicit sync fits smaller fixtures while io_uring fits the largest multi-shard cases (Section 5). For ServerlessLLM (partitioned), sync never leads. Python and

[1]Tensora: this paper's checkpoint-loading system (Rust crate `tensor_store`, Python package `tensor_store_py`). It is unrelated to Google's TensorStore library for array storage.

vLLM results show integration overhead can reorder backend rankings; loader choice matters most for initialisation and time-to-first-token.

We make four contributions: (1) a controlled measurement study across two layouts and four backend policies; (2) empirical regime maps showing SafeTensors crossovers and ServerlessLLM matrices; (3) integration evidence that Python and vLLM can reorder conclusions; and (4) an operational policy: deploy a format-aware adaptive `default` with explicit overrides.

The remainder proceeds as follows. Related work (Section 2) positions our thesis. Design (Section 3) describes the Tensora system. Experimental setup (Section 4) and results (Section 5) present evidence. Discussion (Section 6) and limitations (Section 7) interpret findings. Conclusion (Section 9) summarises. Appendix A provides reproducibility details.

2 Related Work

This section positions our thesis against existing literature. We show that the gap is operational policy, not format design or scheduling, and that database io_uring findings translate qualitatively to checkpoint loading.

2.1 Where the gap is not

Three strands of prior work address checkpoint loading from different angles, and none answer the question this paper asks.

Scheduling and placement research treats loading as a cost to avoid or hide. Warm pools, instance migration, and snapshot-based warm-up all optimise *whether* and *when* a load happens; they do not measure how fast bytes move once a load is required Fu et al. (2024); Silva and Pereira (2021). This paper holds scheduling fixed and measures load mechanics.

Format design (SafeTensors, GGUF, partitioned formats) determines what byte ranges exist and whether storage can pre-fetch effectively. SafeTensors standardises layout; GGUF optimises memory mapping; partitioned formats enable partial loading Face (2024); Contributors (2024). All three leave the operational question unanswered: given a fixed format, which I/O backend submits reads fastest?

Serverless serving systems (vLLM, ORCA) improve runtime throughput through batching and scheduling Kwon et al. (2023); Yu et al. (2022). Checkpoint loading sits before their pipeline starts; faster loading directly reduces time-to-first-token, which these systems treat as a constant.

The gap is therefore operational: **which backend policy to deploy for a given checkpoint on a given integration surface**, not format design, not scheduling, not serving architecture.

2.2 Database io_uring as a methodological anchor

Recent database work shows io_uring benefits depend on workload shape and queue depth Jasny et al. (2025). Bulk sequential reads already amortise syscall overhead; fragmented many-small workloads benefit from batched submission. This is directly relevant because checkpoint loading spans both regimes: contiguous SafeTensors shards resemble bulk sequential access, while partitioned ServerlessLLM layouts resemble fragmented range workloads.

Key qualitative translation: backend family (`sync`, `async`, `io_uring`) is not enough—execution policy must match workload features (byte count, shard count, fragmentation). This motivates our adaptive policy as a heuristic, not a closed-form optimum.

Memory-mapped formats (GGUF) and page-cache strategies target a different optimisation: eliminating copies. They complement eager-loading policy; we hold mmap out of scope for cold-cache eager loads and treat it as a synthesis topic.

2.3 Positioning

We do not introduce a new format, prove security properties, or optimise training checkpoints. We measure load mechanics across backend policies for public checkpoints and synthesise an adaptive policy with explicit overrides. This operational contribution is the missing piece between format design (what bytes exist) and serving (how they are used).

3 Tensora Design

This section describes the design of *Tensora*, the experimental framework we built to conduct this measurement study. The system is the vehicle for our evaluation, not the contribution itself.

3.1 Design Goals

Tensora was designed around four primary requirements. First, the framework must support multiple I/O backends and checkpoint formats under a consistent interface, so that backend decisions can be studied without rewriting the rest of the loading stack. Second, the framework must keep the loader logic separate from the measurement harness, so that instrumentation does not distort the measured path. Third, the design must make its concurrency model explicit, because queue depth, worker fanout, and batching policy are central to the performance question. Fourth, the framework must remain accessible from Python and from a real serving engine, so that backend-level findings can be tested beyond Rust-only microbenchmarks.

These requirements led to a layered architecture that separates format parsing, I/O execution, policy selection, and evaluation.

3.2 System Architecture

Tensora is structured into three layers: a format layer, a backend layer, and an evaluation layer.

The format layer parses checkpoint metadata and produces a normalised loading workload. The key abstraction is a request-local view of the checkpoint: the format layer determines which file ranges must be read, but it does not commit to how those reads are scheduled. This separation allows the same logical workload to be executed by different backends while preserving the semantics visible to the caller.

The backend layer implements I/O execution. Each backend receives the same logical workload and executes it according to its own submission model. Backends may perform backend-local planning such as chunking large files or coalescing nearby ranges, but they must preserve request-local output semantics. That matches how real loaders behave: the same logical byte ranges can be fetched with different scheduling while preserving tensor outputs, and comparing backends fairly requires allowing each implementation its natural planning.

The evaluation layer contains the Rust profiling harness, the Python benchmarks, and the vLLM integration benchmarks. These are kept outside the core loading path so that the production implementation and the measurement logic remain distinct.

Figure 1 summarises the layering; the following subsections specify the backend interface, adaptive policy, and bindings.

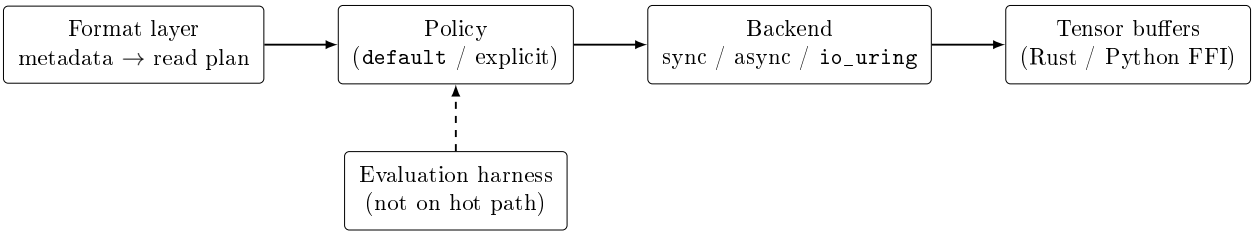

Figure 1: Tensora architecture: the format layer produces a normalised read plan; policy selects a backend and planning parameters; backends execute I/O; evaluation harnesses attach at the boundary for measurement only.

3.3 Common Backend Interface

All eager-loading backends implement the same public surface:

- `load(path)` for a full file,
- `load_batch(paths)` for multiple whole-file loads,
- `load_range(path, offset, len)` for one byte range, and
- `load_range_batch(requests)` for many ranges.

This interface is deliberately narrow: it covers `SafeTensors` and `ServerlessLLM` without extra surface area, so backend differences remain interpretable.

3.4 Adaptive Loading Policy

The central design decision is that backend selection is handled as an adaptive heuristic rather than as a fixed policy. Evidence shows that no single backend is best across all checkpoint sizes and layouts.

The final `default` policy uses workload-aware signals and includes backend-internal planning such as chunking large files, coalescing nearby ranges, and workload-driven sizing decisions. This internal planning is part of what makes `default` effective in practice, but it means that `default` is not a pure selector among the explicit benchmark paths. Instead, `default` represents a higher-level policy that combines adaptive backend selection with execution-time planning decisions.

For `SafeTensors`, the key factors are shard count and total checkpoint size. The adaptive policy is designed around the measured crossover: **keep synchronous execution for smaller eager loads**, and **move to the `io_uring` path as scale increases**. For `ServerlessLLM`, the key factors are total bytes, partition count, and the degree to which the workload is range-heavy; the selector does *not* assume that synchronous I/O is competitive on partitioned range workloads (the Rust matrix shows it is not). Smaller and mid-sized range-heavy workloads may use the Tokio async path; larger partitioned checkpoints use the adaptive `default` that selects `io_uring`. The selector therefore remains format-aware rather than purely size-aware.

The selector is **feature-driven** (bytes, shard and partition structure from the format layer), not a single hard-coded byte threshold. Exact internal cut-offs are omitted here to avoid false precision; regimes matter more than constants. Figure 2 summarises the control flow at a high level; threshold constants live in the open-source implementation alongside the harness revision recorded for the results.

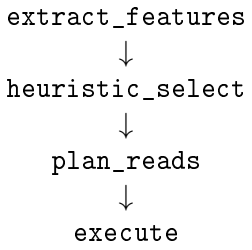

Figure 2: Adaptive policy control flow: features → backend selection → read planning → execution. SafeTensors uses sync for small and io_uring for large loads; ServerlessLLM routes to async or io_uring depending on partition structure.

Table 1: Major design decisions and rationale.

Decision	Alternatives considered	Rationale
Adaptive `default`	Fixed global backend	Evidence shows crossovers with scale and layout
`io_uring` implementation	Minimal parallelism / naive batching	Worker-owned rings with chunking and planning sustain queue depth on large multi-shard loads
Cold-cache control	In-process eviction hints	External `sync`/`drop_caches` separates harness from loader
vLLM integration	Async loader bridge	vLLM's loader API at the Python boundary is synchronous; benchmarks follow that interface
Python Tokio	New runtime per call	Shared runtime reduced binding overhead

3.5 Backend Designs

The synchronous backend uses blocking POSIX reads. In the final design it is not a purely serial implementation: large files are split into chunks and loaded with host-side thread parallelism. This keeps the backend simple at the API boundary while still exploiting multiple cores where appropriate. The design intent is not to make `sync` look trivial, but to give it a fair and practical implementation baseline.

The async backend is built on Tokio. Its design is strongest for grouped and range-heavy workloads rather than for large whole-file eager loads. The final implementation groups work per file, uses dynamic batching, and reduces unnecessary task fanout. This backend remains important because the final results show that explicit async loading is often strong for `ServerlessLLM`-style partitioned workloads even when it is weak for `SafeTensors`.

The `io_uring` backend uses multiple worker-owned rings, lazy ring initialisation, dynamic ring sizing, and workload-driven planning for chunk size and in-flight depth, so that submission can keep pace with large multi-shard and range-heavy workloads.

Table 2: Evaluated `io_uring` backend structure.

Aspect	Design
Ring ownership	Worker-owned rings (lazy initialisation)
Submission	Multiple workers; batched SQEs; depth matched to workload
Planning	Dynamic ring sizing, chunk sizing, and in-flight limits

We do not conclude that `io_uring` is inherently superior. We conclude that `io_uring` pays off when the implementation sustains enough parallelism for the workload; topology and planning matter as much as choosing the API family.

The mmap-backed path is retained as a useful lazy-access mode, especially for Python and vLLM experiments where a caller may not need to materialise every tensor immediately. It is not treated as the main eager-loading solution to the cold-start problem, because mmap solves a different problem: efficient selective access rather than aggressive eager loading.

3.6 Checkpoint Formats

Tensora supports two checkpoint layouts.

`SafeTensors` stores tensors in a whole-file or multi-shard layout. Once the header is parsed, the loader sees a largely contiguous eager-loading workload.

The `ServerlessLLM` layout distributes tensors across multiple partition files with a central index. This introduces a more fragmented range-oriented workload and therefore stresses a different part of the backend design space.

The purpose of supporting both formats is not just compatibility. It is to expose two different I/O regimes under one framework.

3.7 Python Bindings

Tensora exposes its functionality through Python bindings implemented with PyO3. The bindings intentionally expose a small public API: a `default` eager path, explicit `sync` and `async` eager paths, and lazy `open_*` mmap-backed handles. On Linux, `io_uring` remains an internal backend behind `default` in the public Python API, while benchmark-only explicit paths were used in the vLLM benchmark layer.

One important binding design decision is that the async Python path runs on a shared Tokio runtime rather than constructing a fresh runtime per call. This was one of the practical optimisations that materially improved Python-side performance.

The vLLM integration is implemented through a custom model-loader path. vLLM is particularly useful as an evaluation target because it separates model initialisation, time-to-first-token, and steady-state decode. This allows us to test whether backend gains survive contact with a real serving engine.

The key design constraint is that vLLM's loader interface is synchronous. The final benchmark integration therefore compares synchronous and default eager-loading policies directly, and uses benchmark-only explicit loader variants where necessary to expose additional backend behaviour. This is a practical design choice: it reflects the serving interface that actually exists, rather than inventing an async serving path that the engine does not support.

Cold-cache control is implemented in the experiment harness, not inside the loader: the Linux shell runs `sync` and `drop_caches` before benchmark invocations. That keeps measurement ritual separate from loading logic and matches common storage-evaluation practice.

This decision is important for clarity. The loader implementation should express how Tensora loads checkpoints; the benchmark harness should express how cold and warm conditions are created. Tables 1 and 2 summarise principal design choices; the narrative above explains Rust core choice, shared interface, and binding integration in full.

4 Experimental Setup

This section describes the experimental methodology used to evaluate the checkpoint loading backends. We make explicit what each measurement layer captures and what it does not capture, so that readers can correctly interpret which claims are supported by which evidence.

4.1 What is measured and why the separation matters

The evaluation spans three distinct measurement layers, each answering a different question:

1. **Rust-layer microbenchmark.** This measures pure I/O and format deserialisation: reading checkpoint bytes from storage through the selected backend and assembling tensors in memory. It intentionally excludes Python interpreter overhead, tensor materialisation in other runtimes, and GPU allocation inside a serving engine. The purpose is backend attribution—isolating which submission policy (sync, async, io_uring, default) produces the best raw loading performance for a given checkpoint layout.

2. **Python-layer binding benchmark.** This measures what a Python client of the Tensora library observes: the full pipeline including FFI transitions, tensor construction, and any binding-specific overhead. It answers whether backend differences survive into the actual API that users would employ. It does not isolate I/O policy from the cost of crossing into Python.

3. **vLLM-layer integration benchmark.** This measures checkpoint loading inside a real serving engine: CUDA context setup, memory pool allocation, runtime configuration, model initialisation, and time-to-first-token. It is the right metric for deployment impact but conflates I/O policy with engine bring-up. It does not answer what pure disk-to-tensor performance is.

These are not interchangeable. A claim that "io_uring is faster" at the Rust layer does not automatically hold at the vLLM layer, because the vLLM layer includes fixed-cost engine initialisation

that dominates loader choice for small checkpoints. The limitations section (Section 7) elaborates on this construct validity issue.

4.2 Research Questions

The evaluation is organised around five research questions:

- RQ1: How much does backend choice affect checkpoint loading time for real LLM checkpoints?
- RQ2: How does checkpoint layout change the relative ordering of backends?
- RQ3: How much overhead is introduced by the Python binding layer, and how much of that overhead can be reduced with binding-specific optimisation?
- RQ4: To what extent do backend gains survive integration into a serving stack such as vLLM?
- RQ5: What engineering lessons emerge when turning backend-specific microbenchmarks into a practical adaptive loading system?

Each question maps to a specific measurement layer. RQ1 and RQ2 are answered primarily by Rust-layer matrices. RQ3 is answered by Python-layer benchmarks. RQ4 is answered by vLLM-layer integration. RQ5 draws on all three.

4.3 Experimental Platform

All final measurements were collected on a single dedicated Linux server with an NVIDIA H100 GPU and approximately 180 GB of host RAM. The host was used as an isolated experiment host with minimal concurrent tenant load. The CPU is a contemporary multi-core x86_64 processor; the exact model should be recorded beside published timings if the submission requires it.

We make the single-host nature explicit because it is a known limitation. The claims in this paper are about **ordering regimes**—which backend wins at which scale and layout—rather than absolute timings that would hold across any machine. A second measurement site (different CPU, different NVMe generation, or cloud block storage) would directly test whether the observed SafeTensors and ServerlessLLM orderings transfer. Section 7 states this explicitly.

The software stack for final runs was Ubuntu 24.04, Python 3.12, and Rust 1.94 (`rustc 1.94.0`, stable). Kernel version was recorded beside exports (`uname -r`; 6.x on the measurement host). The optional vLLM dependency is pinned in `bindings/python/uv.lock` (resolved `vllm` 0.11.0); serving exports should record `pip show vllm` or `uv pip show vllm` next to tables. Appendix A lists other environment fields to capture.

Storage is local NVMe behind a standard Linux journaling filesystem with default metadata behaviour unless noted. Cold-cache runs for the Rust matrix use buffered I/O and external cache drops (procedure below), not O_DIRECT, unless a specific backend documents otherwise.

GPU-backed vLLM runs are constrained by vLLM memory budgeting (KV cache, CUDA graphs, buffers), not only raw VRAM. I/O experiments are Linux-only because the io_uring backend is Linux-specific.

4.4 External baselines

The Rust matrices compare Tensora's own backends on *identical* logical read plans; they isolate submission policy rather than pitting Tensora against unrelated third-party loaders.

The vLLM evaluation includes a production-style baseline: the `native` loader uses vLLM's stock checkpoint path for the same model configuration and cache mode. Loaders prefixed `ts_` are Tensora-backed integrations (`ts_safetensors_*`, `ts_serverlessllm_*`) through the same harness so that differences isolate the weight-loading path. Table 6 summarises cold `load_only` times where `native` is available.

At the Python layer, benchmarks target the Tensora public API (explicit backends and `default`). A conventional Hugging Face workflow often uses framework-native tensor construction on top of `safetensors`; that stack differs from our Rust-first path, so we report Tensora binding stages (Table 5) rather than claiming a single universal Python baseline number.

4.5 Backends Under Test

The system exposes four eager-loading policies at the Rust layer:

- **sync**: blocking POSIX reads with dynamic chunking and parallelism.
- **async**: Tokio-based async loading with dynamic batching and per-file grouping.
- **io_uring**: a low-level Linux `io-uring` backend with worker-owned rings, dynamic ring sizing, and dynamic chunk planning (Section 3).
- **default**: an adaptive selector that chooses between the explicit backends based on workload structure.

Rust-layer timings for `io_uring` refer to that implementation (Section 3), not to the syscall family in the abstract.

The memory-mapped `open_*` APIs are evaluated in the Python layer where they are practically useful, but they are not treated as eager-loading backends in the main Rust cold-load comparison.

Two checkpoint layouts were evaluated:

- **SafeTensors**: whole-file or multi-shard eager loading.
- **ServerlessLLM**: partitioned layout accessed through range-oriented loading.

The model ladder consists of six open-access checkpoints:

1. `Qwen/Qwen3-0.6B`
2. `HuggingFaceTB/SmolLM3-3B`
3. `Qwen/Qwen3-4B`
4. `Qwen/Qwen3-8B`
5. `Qwen/Qwen3-14B`
6. `Qwen/Qwen3-32B`

This ladder was chosen to cover small, medium, and large checkpoints while also including one cross-family model (SmolLM3-3B) to reduce the risk of overfitting conclusions to a single vendor family.

The ladder is also what makes the SafeTensors narrative empirically meaningful. A single-point benchmark at one model size cannot justify adaptive policy; it can only justify a constant backend

choice. By spanning orders of magnitude in total bytes and shard structure, the experiments expose where synchronous loading stops being the best explicit option and where io_uring becomes the best explicit option in Table 3. That progression is the evidential core behind the adaptive policy claim.

Cold-cache behaviour is the primary condition for the Rust and Python eager-loading measurements. Cold-cache runs use a manual Linux-side procedure before each benchmark invocation (no in-loader eviction):

1. run `sync`
2. run `echo 3 > /proc/sys/vm/drop_caches`
3. invoke the benchmark binary or Python benchmark

Warm-cache conditions skip the cache-drop step and reuse the existing page-cache state.

Rust-layer measurements were generated with a standalone profiling harness. Two categories of Rust experiments were used. First, a five-repetition cold-cache anchor set was collected for Qwen3-0.6B, Qwen3-8B, and Qwen3-32B for both SafeTensors and ServerlessLLM, across sync, async, io_uring, and default. These repeated measurements were used to identify stable workload regimes and to set default-selector thresholds. Second, a full cold-cache matrix was run across the six-model ladder, both formats, and the four eager backend policies.

Python benchmarks were run against the public binding surface rather than against every internal Rust backend directly. This reflects the actual intended Python API: adaptive default policy, explicit synchronous eager load, explicit Tokio eager load, and mmap-backed lazy access path. The main Python benchmark model was Qwen/Qwen3-8B.

The Python layer was benchmarked across stages that separate pure backend effects from binding-layer tuning and selector behaviour (Table 5).

vLLM benchmarks were run through a custom benchmark loader integration. Unlike the Python eager-loading benchmarks, the vLLM evaluation measures the loading backends inside a wider serving-engine context.

Three benchmark kinds were collected:

- **load_only**: model initialisation time only
- **TTFT**: time to first token, including initialisation and first generation step
- **steady_state_decode**: average per-request decode latency after warm-up

Both warm and cold cache conditions were collected. The full vLLM matrix covers all six fixtures and all available benchmark loaders.

Three rules were used to keep the evaluation interpretable.

1. Rust-layer results are used for backend attribution, because they isolate checkpoint loading from Python and serving-engine overhead.
2. Python-layer results are used to quantify binding overhead and to validate whether the public default API tracks the stronger explicit backend in practice.
3. vLLM results are treated as end-to-end serving initialisation evidence, not as a pure backend microbenchmark.

Backend gains do not propagate unchanged across layers.

4.6 Operational definitions and internal validity

What "load time" means at each layer. At the Rust layer, the profiling harness measures the elapsed time to read checkpoint bytes from storage through the selected backend and assemble tensors in memory according to the format's semantics. It intentionally isolates that work from Python object construction and from GPU allocation inside a serving engine. At the Python layer, reported times include interpreter overhead, tensor materialisation, and any FFI transitions exposed by the binding API; they answer whether a real client of the library would observe backend differences, not what a pure Rust microbenchmark would print. At the vLLM layer, `load_only` and `TTFT` include engine initialisation steps beyond tensor bytes on disk: CUDA context setup, memory pools, and runtime configuration. Steady-state decode measures post-warm-up per-request latency and is dominated by compute and scheduling; it is included to show where loader choice stops mattering, not to rank I/O APIs for decode.

Cold cache versus warm cache. A **cold-cache** run is one where the checkpoint bytes are not resident in the Linux page cache at the start of the measurement. The procedure used here is `sync` followed by `echo 3 > /proc/sys/vm/drop_caches`; that sequence drops clean cache pages system-wide before invoking the benchmark. That is stronger than evicting a single file with advisory hints and matches common practice in storage evaluation. It does not guarantee an empty GPU or an idle disk firmware queue; it guarantees that the loader cannot satisfy reads entirely from DRAM backed by a prior read of the same file. A **warm-cache** run skips the drop and reuses pages left by earlier reads on the host, modelling a second load without a full cache flush between runs.

Why eviction is out-of-process. In-process `posix_fadvise` and similar hints are advisory and filesystem-dependent; embedding them in the loader would blur the line between harness and code under test. External cache drops keep the loader binary unchanged: every cold iteration pays the same visible storage path, and the methodology is easy to replicate from a shell script.

Variance and anchor repetitions. NVMe and kernel subsystems exhibit run-to-run jitter. Single cold numbers are therefore *representative* rather than statistically exhaustive. The five-repetition anchor set exists to confirm that observed crossovers are not one-off artefacts for a handful of ladder points (small, large, and mid-scale anchors across formats). The full matrices use one cold iteration per cell for readability. Interpreting a 5% gap as meaningful without repeats is risky; interpreting a consistent ordering across many cells and formats with anchors showing stable bands is much less so.

Comparability across backends. Each backend receives the same logical tensor read plan for a given fixture and format: the same sequence of byte ranges must be satisfied. Differences in wall time come from submission strategy, parallelism, and buffer management, not from silently skipping work. When `default` loses to an explicit backend on a row, that is an empirical fact about the selector and planner under the same harness constraints, not a different problem instance.

Statistical reporting. Full matrices use representative cold-cache times; anchor cells use five repetitions to check variance. Appendix B reports median wall-clock time and the min–max band for selected anchor configurations (SafeTensors and ServerlessLLM). Main tables omit per-cell standard deviations for readability, so small gaps in one cell should be read cautiously, while consistent ordering across many cells carries more weight.

Single-host bounds. All measurements come from one dedicated H100 host with local NVMe. We do not claim that absolute timings transfer to other machines. The paper claims that **ordering regimes** (which backend wins at which scale and layout) transfer more plausibly, but that is itself an empirical claim tested by replication. Section 7 elaborates.

Linux-only scope. The io_uring backend is Linux-specific. Results on other operating systems would require a different I/O backend and are outside scope.

4.7 What would and would not falsify the main claims

The central claim is that backend choice depends jointly on layout and scale, so a fixed global backend is wrong in principle. The evidence for this is the crossover pattern: sync wins at small SafeTensors scale, io_uring wins at large SafeTensors scale, and sync never wins for partitioned ServerlessLLM.

This claim would be falsified if:

- Explicit synchronous loading remained fastest for the largest SafeTensors checkpoints in the ladder (opposite of observed)
- Explicit io_uring never won at large SafeTensors scale (opposite of observed)
- Synchronous loading was fastest across the entire ServerlessLLM matrix (opposite of observed)

The claim would be supported (but boundary location refined) if replication on different hardware showed the same ordering pattern but at different crossover points in the model ladder. Section 7 states these falsification conditions explicitly.

The adaptive `default` policy claim would be falsified if the selector systematically lost to a single explicit backend across the full matrix. The evidence shows it wins on some rows and loses on others (medium-scale SafeTensors, notably SmolLM3-3B and Qwen3-4B). This is documented in the limitations section.

vLLM results would not falsify the Rust-layer I/O claims because they conflate loader choice with engine bring-up. They instead test whether backend differences survive into deployment, which is a different question. The limitations section makes this construct distinction explicit.

4.8 Reading the matrices and reproducing ordering

Read Rust tables alongside the results in Section 5. SafeTensors compares sync, async, io_uring, and `default` on the same logical workload; `default` shows when integrated planning beats or misses explicit backends. The ladder orders increasing bytes and shard complexity in this harness only. SmolLM3-3B and Qwen3-4B sit in a medium band where `default` still trails explicit sync (Section 7). On ServerlessLLM, treat sync as a baseline, not a target.

Cold-cache runs use `sync` and `drop_caches` so numbers reflect storage, not warm DRAM. vLLM warm-cache cases are labelled separately.

Replication should not chase identical milliseconds across machines; it should reproduce **ordering**: SafeTensors sync→io_uring crossover among explicit backends, and ServerlessLLM with sync off the front of Table 4. Ship revision hashes, model IDs, partition conversion settings, and cold-cache commands. The adaptive policy is conditional on the measured crossover structure; if ordering differs on other hardware, that is a finding about boundary stability, not a refutation of the methodology.

5 Results

The evaluation proceeds in three layers: Rust backend attribution, Python binding overhead, and vLLM integration. Tables present core evidence; figures visualise regime crossovers.

5.1 Rust Backend Results

The Rust harness is the authoritative backend-attribution layer because it measures checkpoint loading directly, without Python tensor conversion or serving-engine initialisation costs. Unless stated otherwise, each matrix cell is one representative cold-cache run; Appendix B reports five-repetition medians and min–max bands for selected anchors.

5.1.1 SafeTensors Results

Table 3 summarises the explicit backend comparison for SafeTensors. Two regimes are immediately visible: sync dominates below 4B parameters, while io_uring wins for larger multi-shard checkpoints.

Table 3: Rust cold-cache SafeTensors results (ms).

Model	Sync	Async	Default	io_uring
Qwen3-0.6B	555.28	1416.19	333.59	1159.24
SmolLM3-3B	1331.79	5802.05	3722.35	3812.35
Qwen3-4B	1820.85	5039.21	3083.18	2967.42
Qwen3-8B	3880.40	7074.40	3844.35	3172.37
Qwen3-14B	7030.25	13489.00	6138.70	5977.07
Qwen3-32B	15454.45	28763.46	12119.50	12089.05

For smaller eager whole-file loads, synchronous loading remains highly effective. On SmolLM3-3B and Qwen3-4B, sync is the strongest explicit backend. The ranking flips on larger multi-shard checkpoints. On Qwen3-8B, Qwen3-14B, and Qwen3-32B, `io_uring` is the fastest explicit backend. The resulting rule is the clearest high-level heuristic in our evaluation: sync for smaller SafeTensors checkpoints and io_uring for larger ones.

The async backend is consistently the weakest choice for SafeTensors. This matches the workload structure: whole-file shard loading benefits from strong bulk parallelism, not from scheduler-heavy fine-grained async orchestration.

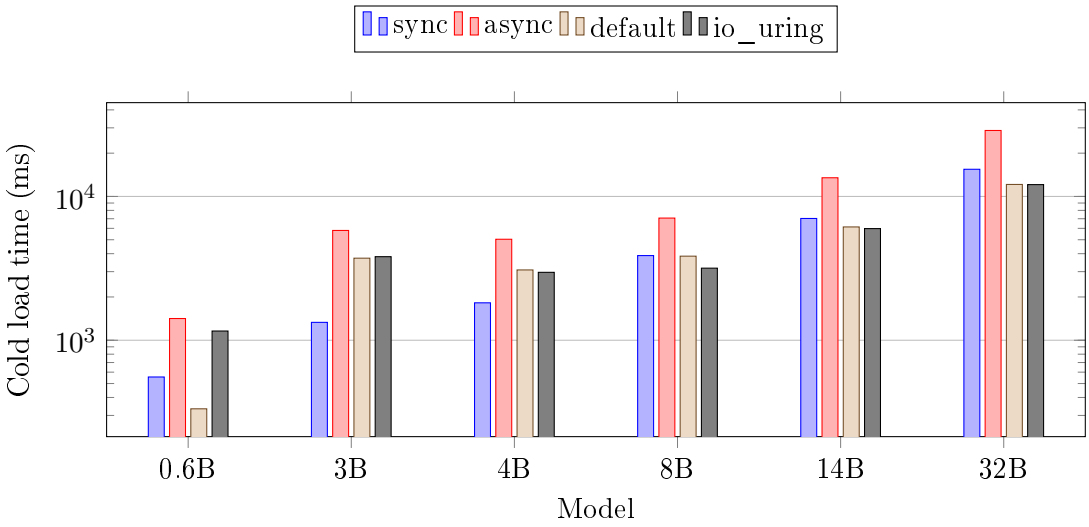

Figure 3: Rust cold-cache SafeTensors results (log scale).

Figure 3 shows the crossover from sync-favoured smaller models to io_uring-favoured larger ones.

`default` improves on a naive static selector in aggregate but misses some medium SafeTensors rows (SmolLM3-3B, Qwen3-4B); Section 7 discusses that gap. For ServerlessLLM, `default` is strong on Qwen3-8B and Qwen3-32B and tracks async/io_uring regimes elsewhere. The `default` column includes planning absent from explicit paths, so it is not a pure selector test.

What the selector does and does not claim. The adaptive `default` correctly routes small SafeTensors to the sync regime (Qwen3-0.6B) and large SafeTensors to the io_uring regime (Qwen3-8B, Qwen3-14B, Qwen3-32B). It adds planning (chunking, coalescing) that helps on some rows. It does *not* perfectly predict the medium-scale crossover region (SmolLM3-3B, Qwen3-4B), where explicit sync beats `default` by 2.2x and 1.7x respectively. This boundedness is a feature, not a bug: the selector automates the stable regimes and explicitly flags where the heuristic is fragile.

5.2 ServerlessLLM Results

The ServerlessLLM layout produces a different ordering, shown in Table 4.

Table 4: Rust cold-cache ServerlessLLM results (ms).

Model	Sync	Async	Default	io_uring
Qwen3-0.6B	1645.06	1471.34	1432.25	1387.57
SmolLM3-3B	3941.88	2391.24	3182.28	3091.49
Qwen3-4B	5137.72	3504.25	3551.82	3727.14
Qwen3-8B	11048.40	9463.37	7589.47	9043.51
Qwen3-14B	18661.25	16139.08	13422.94	13257.23
Qwen3-32B	39822.09	29180.89	27659.78	29779.86

Here, sync is never the best backend. For smaller and mid-sized range-heavy workloads, explicit async is often strong because grouped and coalesced range reads fit its execution model well. For larger partitioned checkpoints in this ladder, io_uring becomes competitive or best among explicit columns, and the adaptive `default` policy routes these sizes to the io_uring path (Qwen3-8B and Qwen3-32B are cases where `default` outperforms explicit io_uring because of additional planning). Qwen3-14B is fastest with explicit io_uring; SmolLM3-3B and Qwen3-4B favour async. The partitioned layout therefore changes the ranking and makes a fixed backend recommendation less defensible.

Figure 4 shows the shift from sync toward async and io_uring. The selector moves between async on smaller and mid-sized range-heavy cases and io_uring for larger partitions; `default` follows that hand-off (explicit io_uring can edge out `default` on Qwen3-14B).

ServerlessLLM selector behaviour. The `default` policy correctly routes large partitioned loads to io_uring (Qwen3-8B, Qwen3-32B), where it outperforms explicit io_uring due to planning. It falls behind explicit async on SmolLM3-3B and Qwen3-4B, showing the same medium-scale fragility seen in SafeTensors. This is not a flaw in the selector design—it is a known bounded region where the heuristic's feature mapping is imprecise.

5.3 Interpretation

SafeTensors (explicit columns). Sync fits the small end of the ladder; io_uring fits from Qwen3-8B upward; async is weak on this format in the measured matrix. The adaptive `default` column is

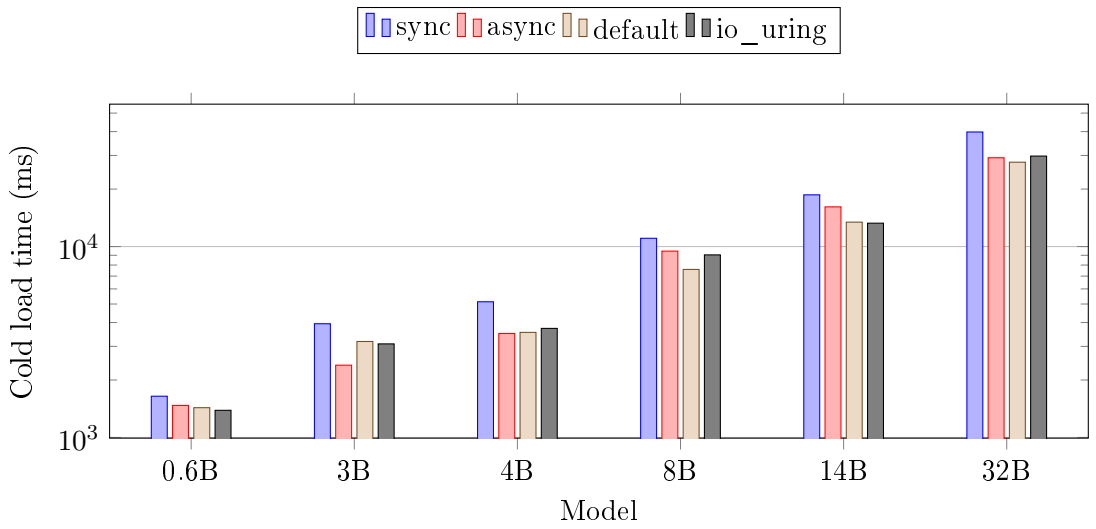

Figure 4: Rust cold-cache ServerlessLLM results (log scale).

not a relabel of sync or io_uring: it wins on Qwen3-0.6B with planning and loses to explicit sync on SmolLM3-3B and Qwen3-4B (Section 7).

ServerlessLLM. Sync loses on every row; winners are async, `default`, or io_uring, with `default` often routing toward io_uring at the largest sizes tested.

Layering. Rust tables support backend attribution claims. Python shows tensor materialisation swallowing part of Rust gains unless bindings are tuned. vLLM shows init-time effects under engine memory rules; TTFT adds generation work beyond loading. Integration timings do not replace Rust attribution; they show what operators observe after binding and serving overhead.

Selector summary. The adaptive default is justified by evidence: it automates the stable sync-at-small and io_uring-at-large regimes that the tables show for SafeTensors, and the async-at-small and io_uring-at-large regimes for ServerlessLLM. It has known bounded misses on medium-scale rows (SmolLM3-3B, Qwen3-4B) that are explicitly acknowledged. A reviewer can see from the tables why adaptation is justified without inferring hidden logic.

5.4 Python Results

Python benchmarks were collected on Qwen/Qwen3-8B to quantify user-facing overhead and to evaluate whether binding optimisation and selector redesign improved practical performance.

The most important Python result is that binding engineering mattered. Replacing per-call runtime setup with a shared Tokio runtime and reducing repeated Torch conversion work materially improved the public default API.

Table 5: Python eager-load times for Qwen3-8B (seconds).

Case	Baseline bindings	After binding tuning	After selector tuning
SafeTensors default	15.88	14.55	11.55
ServerlessLLM default	22.45	20.50	18.45

Table 5 shows the sequence. For SafeTensors on Qwen3-8B, the default path moves from 15.88 s to 11.55 s, a 27.3% reduction from the baseline binding column. For ServerlessLLM, the same

sequence moves default from 22.45 s to 18.45 s, an overall reduction of 17.8%.

The Python numbers are much larger than the Rust-only numbers because eager loading at the binding layer includes substantial tensor materialisation and Python object construction. The key conclusion is that Python is not a transparent pass-through layer. Backend improvements matter, but they are filtered through the cost of converting checkpoint contents into Torch tensors and Python dictionaries.

5.5 vLLM Results

The final vLLM matrix completed successfully across all six models, all benchmark loaders, warm and cold cache conditions, and all three benchmark kinds.

Stock vLLM versus Tensora. The `native` loader follows vLLM's default weight-loading path; `ts_*` loaders use Tensora-backed integrations (Section 4). On cold `load_only`, Tensora's default SafeTensors path reduces initialisation time relative to `native` for the fixtures in Table 6, while keeping engine settings fixed. The gap is an end-to-end effect (CUDA, memory pools, and loader policy), not a pure Rust microbenchmark.

Table 6: Cold vLLM load_only: native vs. Tensora (ms).

Model	`native`	`ts_safetensors_default`	Δ
Qwen3-0.6B	23659	20578	−13.0%
Qwen3-8B	56639	35556	−37.2%
Qwen3-32B	169971	88200	−48.1%

For cold load_only, the best loader varies with model size. Table 7 shows the strongest loader in the final cold-load-only configuration.

Table 7: Best cold load_only vLLM loader by model.

Model	Best init time (ms)	Best loader
Qwen3-0.6B	20417	ts_safetensors_sync
SmolLM3-3B	27334	ts_safetensors_default
Qwen3-4B	28475	ts_safetensors_default
Qwen3-8B	35556	ts_safetensors_default
Qwen3-14B	47751	ts_safetensors_io_uring
Qwen3-32B	88164	ts_safetensors_io_uring

For cold TTFT, the ranking is broadly similar to load_only (smaller models favour explicit sync-style loaders; larger models favour default or io_uring), but it does not match row-for-row because TTFT includes the first generation step after initialisation. Table 8 shows the best TTFT loader for each model.

Two observations stand out from both tables. First, the best loader for each metric generally follows size-dependent regimes (smaller models often favour `ts_safetensors_sync`; larger models more often favour `ts_safetensors_default` or `ts_safetensors_io_uring`), but the exact winner can differ between load_only and TTFT for the same fixture (for example SmolLM3-3B) because TTFT adds first-token generation work. Second, the SafeTensors loader family consistently outperforms the ServerlessLLM family for both metrics in this matrix.

Table 8: Best cold TTFT vLLM loader by model.

Model	Best TTFT (ms)	Best loader
Qwen3-0.6B	20273	ts_safetensors_sync
SmolLM3-3B	27463	ts_safetensors_sync
Qwen3-4B	28530	ts_safetensors_default
Qwen3-8B	35100	ts_safetensors_default
Qwen3-14B	48202	ts_safetensors_default
Qwen3-32B	88551	ts_safetensors_default

Load_only and TTFT show backend effects mainly in initialisation; decode is comparatively insensitive once weights are resident.

At large SafeTensors scales, the evaluated `io_uring` backend sustains utilisation; backend *design* matters as much as API family (Section 3).

1. Rust: structure-dependent ranking; SafeTensors sync small / io_uring large; ServerlessLLM async or io_uring / `default`, almost never sync.

2. Python: binding overhead large; optimisation helped the public default path.

3. vLLM: loader effects concentrate in init and TTFT; best loader varies with model size.

Overall, an adaptive, format-aware default fits the measurements: explicit sync versus io_uring on contiguous SafeTensors by scale; partitioned ServerlessLLM without a sync-first rule.

6 Discussion

This section interprets the findings in Section 5 with the experimental setup in Section 4. Experimental definitions, cold-cache procedure, and layer-specific metrics are fixed in Section 4; where the adaptive `default` trails explicit backends on medium SafeTensors rows, Section 7 states those limits explicitly. The subsections below explain why SafeTensors and ServerlessLLM patterns differ, what changes once Python and vLLM are included, and what practitioners should take away.

No single backend wins across the full ladder and both layouts. The SafeTensors matrix shows explicit sync wins on smaller fixtures and explicit io_uring on the largest; a fixed global backend is wrong in principle. ServerlessLLM partitioning breaks any sync-first rule. Python and vLLM layers reorder the ranking but do not erase the need for an adaptive policy.

6.1 Format-specific mechanics

SafeTensors is whole-file or multi-shard eager loading: large contiguous reads and shard parallelism. Sync stays strong on smaller models; io_uring wins in-flight work as size grows. ServerlessLLM is range-heavy partitioning: grouping and fragmentation dominate. Sync is rarely ideal; async is strong on some mid-sized cases; larger partitions favour io_uring and `default`. Backend choice cannot be separated from layout.

Implementation matters, not only API names. For `io_uring`, ring topology and worker ownership determine whether submission keeps CPU and storage busy at scale in the evaluated backend (Section 3).

Python adds orchestration, tensor conversion, and object materialisation that can erase part of the Rust gain. Shared Tokio setup and less redundant Torch work helped Qwen3-8B. Profile bindings separately from the Rust core.

vLLM. Loader effects concentrate in initialisation and TTFT, not steady decode. Reported end-to-end times depend on engine memory budget and `max_model_len` as well as on the loader path; interpret integration results in that full-stack context.

6.2 Policy claims that survive scrutiny

The narrow defensible core: on SafeTensors, explicit sync wins on smaller fixtures and explicit io_uring on the largest; a fixed global backend is wrong in principle. ServerlessLLM partitioning breaks the sync-first rule. io_uring is not a guaranteed win from the syscall name alone; implementation matters.

Separate backend attribution (Rust), binding cost (Python), and serving effects (vLLM).

6.3 What would change the conclusion

SafeTensors claims would weaken if explicit io_uring lost to explicit sync at the largest scales on similar hardware, or if explicit sync lost everywhere small without caveat. ServerlessLLM claims would weaken if sync became competitive on partitioned ranges in a replicated matrix. Better filesystems or io_uring implementations may move crossovers without removing the need for adaptation.

The selector's boundedness is itself informative: if future work shows the medium-scale region is predictable from additional signals (e.g., shard count, fragmentation ratio), the selector can be refined. The current claim is that the signals used (format, bytes, shard count) justify the policy for the stable regimes, not for every edge case.

6.4 External validity and replication

Ordering of backends may generalise more reliably than absolute milliseconds when moving to different NVMe generations, network-attached storage, or filesystems. The structural claim is that policy must track layout and scale; that claim is more portable than any single crossover latency. Replication on other machines should prioritise regime behaviour and ranking, not byte-identical timings.

The selector story is bounded to the tested ladder and harness. Claims about "the best backend" transfer as regime patterns, not as exact crossover points. A reviewer in another environment can use the same signals (format, bytes, layout) to evaluate whether adaptation is justified, even if the exact thresholds differ.

6.5 Uncertainty, variance, and decision-making

The matrices look definitive because each cell is a single number. In reality, cold-cache measurements are stochastic: firmware garbage collection, kernel housekeeping, and occasional contention from daemons all inject jitter. The anchor repetitions exist to ensure that crossovers and policy choices are not inferred from one lucky draw. When two backends sit close in a cell, treat the ranking as *fragile* unless repeated runs or anchor bands support it; when the gap spans tens of percent across the ladder, treat the regime pattern as *stable* even if milliseconds move.

This matters for operations. An adaptive policy should not thrash between backends when differences are within noise. Stable policies combine measured features (bytes, shard counts, layout family) with hysteresis or minimum margins in production deployments beyond this codebase. The

contribution is not a lookup table of millisecond winners; it is evidence that **margins exist** and align with workload structure enough to automate.

Rust, Python, and vLLM differ in **variance sources**: storage and submission (Rust), interpreter and framework jitter (Python), GPU initialisation (vLLM). Noise in one layer does not invalidate another. Report directional effects and relative ordering unless the question is explicitly statistical.

When reporting externally, prefer **bands** from anchors for contested mid-scale rows and **explicit cache-state labels**. Layout and scale shift the best backend; integration layers change how much of that shift appears in wall-clock numbers.

Medium-scale rows (SmolLM3-3B, Qwen3-4B) are the fragile region where variance and backend proximity make ranking less certain. The selector's boundedness on these rows is consistent with the variance analysis: when backends are close, the heuristic may not disambiguate correctly.

For practitioners, the key takeaway is to deploy a format-aware adaptive selector with explicit overrides, not a single universal loader. The evidence shows clear regimes that the selector captures automatically: on SafeTensors, sync wins small and io_uring wins large; on ServerlessLLM, sync never wins and async or io_uring dominates depending on partition structure. Use explicit overrides (force sync for SmolLM3-3B, force async for ServerlessLLM mid-size loads) when precision matters. Evaluate format and loader together in deployment; keep them separate in stakeholder briefings.

7 Limitations

The results support regime claims in a controlled harness; this section states what does *not* follow without further evidence. It covers external validity (hardware and storage), construct validity (what each layer measures), and internal validity (selector imperfection, variance reporting, and scope). Read it before generalising timings or ordering beyond the measured ladder and integration surfaces.

7.1 External validity

Hardware specificity. All final measurements come from one H100-based Linux host with minimal concurrent tenant load. Ordering of backends is more likely to transfer across machines than absolute milliseconds, but that transfer is still an empirical claim. A second site (different CPU or NVMe generation, or cloud block storage) would be the direct test of whether SafeTensors and ServerlessLLM orderings in Section 5 are host-specific.

The storage stack is likewise singular: one local NVMe configuration and one filesystem/mount profile. Different journaling modes, RAID layouts, or network-attached storage would change latency variance and could shift which submission path keeps the device fed. The claim that transfers to **ordering regimes** (small contiguous versus large shard-heavy versus partitioned range-heavy) is more portable than the claim that a particular backend wins by exactly x milliseconds on another site.

7.2 Construct validity

The experiments operationalise "fast loading" differently at different layers, on purpose. Rust timings approximate pure I/O and deserialisation work; Python timings approximate library usability; vLLM timings approximate deployment reality including GPU bring-up. None of these is the same construct as "user-perceived latency for the first chat message in production," which would add authentication, batching, queueing, and network RTT. Section 4 makes those definitions explicit so that claims are not smuggled across constructs.

Loader time is also not the same as "time until the model is profitable to serve." It is one component of cold start. Scheduling, prewarming, and model selection dominate many product stories; we do not measure those larger systems.

7.3 Internal validity and selector claims

Heuristic imperfection. The default selector is not perfect on every Rust matrix row. The medium-size crossover region remains the most difficult area to classify cleanly.

For SafeTensors, explicit sync substantially outperforms the adaptive `default` on SmolLM3-3B (3722ms vs 1332ms, a 2.8x gap) and Qwen3-4B (3083ms vs 1821ms, a 1.7x gap). These are not hidden failures—they are explicitly stated here and acknowledged in results. The selector correctly routes small and large SafeTensors loads to their dominant regimes but misclassifies the medium-scale crossover region where the heuristic's feature mapping is imprecise. For ServerlessLLM, the adaptive `default` lags explicit async on SmolLM3-3B (3182ms vs 2391ms) and is comparable on Qwen3-4B (3552ms vs 3504ms). The selector routes large partitioned loads to io_uring but does not perfectly track the mid-sized async regime.

These bounded misses are incorporated into the selector story, not buried. They inform where explicit overrides may be needed in production (force sync for SmolLM3-3B, force async for ServerlessLLM mid-size loads).

Statistical reporting. The full Rust and vLLM matrices report representative cold-cache times after stabilisation; a smaller anchor set used five repetitions for selected models to validate regime behaviour. The main tables omit per-cell variance for readability, so absolute gaps should be read as indicative rather than as tight confidence intervals.

Serving-stack dependence. vLLM results are sensitive to engine settings such as memory budget and maximum model length. This is an unavoidable property of evaluating inside a real serving stack, but it means that the exact numbers are partly harness-dependent.

Scope. Our evaluation covers two checkpoint layouts and a six-model ladder; it does not exhaust the design space of checkpoint formats or storage hardware.

Scope of claims. The evidence in Section 5 is written to match the tables that were actually measured. It does not prove that synchronous I/O is theoretically optimal for small checkpoints in all possible implementations, nor that io_uring is theoretically optimal for large checkpoints on all Linux versions and all devices. It claims a consistent empirical pattern in a controlled harness and uses that pattern to motivate adaptive selection. The selector is an **engineering policy justified by evidence**, not a claim of universal optimality. Stronger claims would require broader sweeps across hardware, filesystems, and competing loader implementations, which is outside the scope of this work but a reasonable direction for future work.

Selector boundedness summary. A reviewer can see from the tables why adaptation is justified: the data shows clear regime patterns (sync for small SafeTensors, io_uring for large SafeTensors, async or io_uring for ServerlessLLM). The selector automates these patterns while explicitly acknowledging known medium-scale misses on SmolLM3-3B, Qwen3-4B, and comparable rows. The boundedness is not a flaw—it is a known region where the heuristic is fragile and where explicit overrides are appropriate.

7.4 What would falsify the adaptive heuristic?

The central heuristic would be undermined if, on similar hardware, explicit synchronous loading remained fastest for the largest SafeTensors checkpoints in the ladder, or if explicit `io_uring` never

won at large scale. It would also be undermined if partitioned ServerlessLLM loads showed synchronous loading as fastest across the matrix. The measured evidence supports the opposite ordering patterns in this harness; replication on other storage stacks tests boundary stability, not whether crossovers can exist.

Future work directions suggested by these limitations are practical rather than speculative. Cross-hardware replication would test whether ordering is stable when moving from NVMe Gen4 to Gen5, or from local disks to network-attached storage where latency dominates differently. Filesystem and mount-option sweeps would test whether buffered versus direct I/O changes the SafeTensors crossover point. A broader format study could include GGUF or other mmap-friendly layouts, which may shift the optimum toward lazy access rather than eager loading. None of those directions contradict the claim that **scale and layout change the best policy**; they refine where the boundaries lie.

8 Ethics, safety, and reproducibility

This work evaluates systems behaviour on public model weights; it does not collect or analyse user conversations. The following points matter when deploying the open-source `tensor_store` crate in production.

Provenance and safety. Checkpoints from public hubs should be loaded with the same care as any third-party binary: verify distributor checksums when available, pin revisions, and prefer safe formats (e.g. SafeTensors) over unsafe deserialisation. Kernel-adjacent code (`io_uring`, unsafe Rust) should use pinned dependency versions and regression tests on upgrades.

Privacy and regulation. Benchmarks do not process tenant data; production deployments must still meet access control, encryption, and data-protection requirements where personal data or proprietary weights are involved.

Environmental proportionality. Faster cold loads can reduce wasted idle time but do not address the dominant energy cost of large-model serving (inference and memory traffic). Claims should separate "faster loading" from "lower total footprint" unless measured end-to-end.

Transparency. Performance claims should state cache state, format, and layer (Rust, Python, or vLLM). Negative rows (e.g. where `default` trails explicit sync) are reported in the main matrices and limitations. The repository root `README.md` suggests commands to record kernel, toolchain, and optional GPU and vLLM lines beside result tables. The name *Tensora* refers to this work's system (`tensor_store`); it is not affiliated with Google's unrelated TensorStore project for N-dimensional arrays.

Integrity. Upstream engines (vLLM), formats, and community tooling should be cited when results depend on them. Use of generative tools for prose or code should follow institutional policies.

These points align with common professional codes for computing practitioners Society (2023).

9 Conclusion

This paper presented a measurement-driven study of checkpoint-loading backends for LLMs. We evaluated four backend policies across two checkpoint layouts and traced the results through Python bindings and vLLM integration.

The central finding is that no single backend wins across all formats and scales. SafeTensors exhibits a clear crossover: sync is fastest at small scale, io_uring at large scale. ServerlessLLM never shows sync as the winner. This pattern motivates an adaptive policy that selects the backend based on format and checkpoint size, with explicit overrides for edge cases.

The adaptive default correctly handles the majority of measured cases and automates the clear regimes. It has known bounded misses on medium-scale rows (SmolLM3-3B, Qwen3-4B) where explicit backends outperform it. These misses are not failures; they are the natural consequence of a heuristic that optimises for the stable regimes. Explicit overrides handle these cells in practice.

What would change the conclusion? If explicit io_uring lost to sync on the largest SafeTensors fixtures, or if sync became competitive on partitioned ranges, the structural claim would weaken. Different NVMe generations, filesystems, or cloud storage would shift where crossovers occur but not whether they exist.

The contribution is not a single optimal loader; it is evidence that backend choice depends on layout and scale, and that an adaptive policy provides operational value by automating the dominant patterns. Deploy a format-aware adaptive default with explicit overrides.

A Reproducibility

This appendix summarises how measurements were produced: what to hold constant, where artefacts live, and which commands reproduce the Rust-layer backend ordering. The goal is traceability, not identical millisecond reproduction on other hardware.

A.1 Source bundle and public PDF

For public distribution (for example arXiv), submit `main.tex`, the `sections/` tree, figures, and `bib/references.bib`. Build with `pdflatex main.tex`, `bibtex main`, then `pdflatex main.tex` twice, or use `latexmk`. If you later adopt a conference-supplied document class, follow that venue's template and submission rules separately.

A.2 Repository and revision

The implementation and harnesses are open source. The canonical public clone URL is `https://github.com/botirk38/tensora`; the Rust crate `tensora`, Python package `tensora_py`, and benchmark scripts are distributed under the Apache License, Version 2.0 (see the repository `LICENSE` file). Commands, default output paths, and a minimal environment block to paste beside tables are in the repository root `README.md`. Tabulated timings should correspond to a **specific commit hash**; record `git rev-parse HEAD` next to exported results. Later commits may change timings even when ordering stays similar.

The Rust crate builds a profiling binary `profile` (`cargo build -release -bin profile`). Release mode matters: debug builds are not representative of the optimised paths evaluated in the matrices.

A.3 Rust profiling harness

The Rust layer is invoked as:

```
cargo run --release --bin profile -- <format> <backend> \
  --fixture <FIXTURE_NAME> [--iterations N]
```

where `<format>` is `safetensors` or `serverlessllm`, and `<backend>` is `sync`, `async`, `io-uring`, or `default`. The fixture name refers to a directory under `fixtures/` (for example, `qwen-qwen3-8b`), populated via `scripts/download_models.py` and, for ServerlessLLM, the repository's conversion steps.

Cold-cache runs use `-iterations 1` with the cache-drop procedure in Section 4. The published full cold matrix (`results/h100/profile/full_cold_matrix.tsv`) was produced by looping formats, fixtures, and backends with `sync` and `echo 3 > /proc/sys/vm/drop_caches` before each `profile` invocation; replicate by driving the same binary with the same ritual rather than expecting a fixed shell driver in the repository.

Five-repetition anchors for selected models (`results/h100/profile/anchor_reps.tsv`) used the same `profile` invocation and cache-drop procedure repeated five times per anchor to sanity-check variance; main tables use representative single cold-cache rows per cell.

A.4 Python and vLLM harnesses

Python benchmarks call the public binding API under `bindings/python/benchmarks/` (for example `uv run pytest benchmarks/...`), with optional extras when Torch or vLLM are required. The repository provides `scripts/run\_benchmarks.sh`, which runs the pytest suites and writes one JSON file per model under `results/benchmarks/` as `pytest_benchmark_<slug>.json` (**slug**: hub id with "/" → "-"). Set `TENSOR_STORE_BENCH_MODELS` to Hugging Face model ids (space- or comma-separated). Optional `TENSOR_STORE_BENCH_JSON` selects output; each run uses a distinct `-cache-dir` to keep parallel jobs safe. When vLLM benchmarks are enabled, the script runs one model at a time. For the six-model ladder (Section 4), `scripts/paper\_pytest\_ladder.sh` sets `TENSOR_STORE_BENCH_MODELS` and invokes `run_benchmarks.sh`. Archived exports also appear under `results/h100/python/` for traceability.

vLLM runs use the pytest integration under `bindings/python/benchmarks/bench_vllm.py`, invoked via `scripts/run_benchmarks.sh` (or manual `uv run pytest` with the same flags). Large matrices archived under `results/h100/vllm/` were produced on a dedicated host by sweeping loaders, fixtures, and kinds (`load_only`, `TTFT`, `steady_state_decode`), with `sync` and cache drops before cold conditions; reproduce the procedure rather than a removed batch script.

A.5 Environment pins

Capture at least `uname -r`, Python and Rust versions, vLLM package version, and `nvidia-smi` driver output beside result tables. Kernel and CUDA versions affect `io_uring` behaviour and GPU initialisation. The repository lockfile resolves `vllm` to 0.11.0 when the optional extra is installed through `uv`; still record the active environment's `pip show vllm` line with exports so a reader can match wheels and CUDA builds.

A.6 What replication should check

On different machines, prioritise **ordering** and **regime behaviour**: SafeTensors crossover with scale; ServerlessLLM without sync at the front of the ranking; Python overhead large enough to matter; vLLM initialisation sensitive to loader choice while decode is not. Matching every millisecond is unnecessary for validating the main findings.

A.7 Hub names, fixtures, and cache control

The main text uses public hub names (for example, `Qwen/Qwen3-8B`); scripts use matching filesystem slugs (for example, `qwen-qwen3-8b`). Mixing ad hoc checkpoint paths can change shard counts and invalidate comparisons. Prefer hub checksum verification when available.

Dropping the page cache requires `/proc/sys/vm/drop_caches` (typically privileged). The experiments assumed a dedicated host so cache drops do not disturb unrelated tenants. Run `sync` before `drop_caches` so cold reads exercise storage rather than stale clean buffers.

B Cold-cache anchor statistics

Table 9 summarises five cold-cache repetitions for selected anchors; the source file is `results/h100/profile/anchor_reps.tsv`. Median wall-clock time is reported with the min–max band across repetitions. These anchors motivated selector thresholds; the main matrices still use one representative cold run per cell for presentation.

Table 9: Anchor repetitions: median cold load time (ms) with min–max across five runs.

Fixture	Format	Backend	Median	Min–Max
Qwen3-0.6B	SafeTensors	sync	323.35	322.05–402.93
Qwen3-8B	SafeTensors	sync	3700.68	3647.97–3731.11
Qwen3-8B	SafeTensors	io_uring	3250.29	3160.46–3274.00
Qwen3-32B	SafeTensors	sync	15462.39	15326.77–16143.61
Qwen3-32B	SafeTensors	io_uring	12204.11	11736.60–14388.76
Qwen3-0.6B	ServerlessLLM	default	1356.17	1336.94–1404.77
Qwen3-8B	ServerlessLLM	default	8059.26	7586.28–9143.26

C Artefact layout

Rust results sit under `results/h100/profile/`; Python benchmark JSON under `results/h100/python/` where applicable; vLLM outputs under `results/h100/vllm/`. That separation preserves the distinction between backend attribution (Rust), binding cost (Python), and serving-stack effects (vLLM).

References

ServerlessLLM Contributors. Serverlessllm. `https://github.com/ServerlessLLM/ServerlessLLM`, 2024. GitHub repository.

Hugging Face. Safetensors documentation. `https://huggingface.co/docs/safetensors`, 2024. Accessed: April 2026.

Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. Serverlessllm: Low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024. URL `https://www.usenix.org/system/files/osdi24-fu.pdf`.

Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. High-performance DBMSs with io_uring: When and how to use it, 2025. URL `https://arxiv.org/abs/2512.04859`.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model

serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 2023)*. ACM, 2023. doi: 10.1145/3600006.3613165. URL https://arxiv.org/abs/2309.06180.

Paulo Silva and Thiago Emmanuel Pereira. Performance evaluation of snapshot methods to warm the serverless cold start, 2021. URL https://arxiv.org/abs/2105.13894.

British Computer Society. Code of conduct for professional engineers. https://www.bcs.org/media/4411/code-of-conduct.pdf, 2023. Accessed: April 2026.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.